

Big Data I
(6 ECTS)

Course Manual

dr. Anne Kroon
Graduate School of Communication
University of Amsterdam
a.c.kroon@uva.nl

Academic Year 2026/27
Semester 1, block 2

Chapter 1

About this course

This course manual contains general information, guidelines, rules, and schedules for the Research Master course Big Data & Automated Content Analysis Part I (6 ECTS). Please read it carefully, as it contains information regarding assessments, deadlines, and grading.

1.1 Course description

“Big data” refers to data that are more voluminous, but often also more varied, unstructured, and dynamic than data traditionally used in communication research. In Communication Science and the Social Sciences more broadly, this includes research using internet-based data sources such as social-media content, digital news archives, platform data, and public comments. This area of research is commonly described as *Computational Social Science* (Lazer et al., 2009) or, when focused on communication processes and content, *Computational Communication Science* (Shah, Cappella, & Neuman, 2015).

This course introduces the concepts, opportunities, and challenges of computational research with digital communication data. Computational approaches can make it possible to analyse large or complex collections of texts and other digital traces; they complement rather than replace established communication-science methods such as theory-driven research design, manual content analysis, and statistical inference. Throughout the course, we emphasise that automated outputs require substantive interpretation and empirical validation.

Participants learn practical strategies for acquiring, organising, preprocessing, analysing, and doc-

umenting digital data in communication contexts. The course focuses on (a) data acquisition from files and APIs, data wrangling, exploratory analysis, and reproducible workflows; and (b) automated content analysis, including text preprocessing, dictionary and Bag-of-Words approaches, unsupervised text analysis, embeddings, topic modelling, and supervised machine-learning classification. Students learn to inspect and evaluate automated outputs, compare them with appropriate benchmarks, and reflect on validity, ethics, reproducibility, and the responsible use of AI tools.

The course uses Python as its main programming environment. Students are expected to be willing to learn to write, read, adapt, and explain Python code. Previous programming experience is helpful but not required. Students without prior experience are encouraged to complete one of the many free online introductions to Python before the course begins.

1.2 Goals

Upon completion of this course, the following goals are reached:

3. Knowledge and Understanding: Have in-depth knowledge and a thorough understanding of advanced research designs and methods.

3.1. Have in-depth knowledge and a thorough understanding of advanced research designs and methods, including their value and limitations.

3.2. Have in-depth knowledge and a thorough understanding of advanced techniques for data analysis.

A Students can explain *fundamental research designs* and *commonly employed methods* in existing research articles on Big Data and automated content analysis.

- B Students can on their own and in own words critically discuss the pros and cons of *fundamental* research designs and methods employed in existing research articles on Big Data and automated content analysis; they can, based on this, give a critical evaluation of the methods and, where relevant, give advice to improve the study in question.
- C Students are able to identify some *basic* techniques from the field of computer science and computer linguistics that are applicable to research in communication science; they can explain the principle of some traditional approaches to text analysis, namely simple rule-based techniques and basic methods of unsupervised and supervised machine learning.
4. Skills and abilities: Are able, independently and on their own, to set up, conduct, report and interpret advanced academic research.
- 4.1 Are able to formulate research questions and hypotheses for advanced empirical studies.
- 4.2 Are able to develop a research plan, choose appropriate and suitable research designs and methods for advanced empirical studies, and justify the underlying choices.
- 4.3 Are able to assess the validity and reliability of advanced empirical research, and to judge the scientific and professional value of findings from advanced empirical research.
- 4.4 Are able to apply advanced empirical research methods.
- D Students can on their own formulate a research question and hypotheses for own empirical research in the domain of Big Data.
- E Students can on their own chose, execute and report on *fundamental* research methods in the domain of Big Data and automatic content analysis.
- F Students know how to collect data with APIs or read in existing data files; they know how to analyze these data with fundamental automated techniques and to this end, they have basic knowledge of the programming language Python and know how to use fundamental Python-modules for communication science research.
6. Academic attitudes
- 6.1 Regularly assess their own assumptions, strengths and weaknesses critically.
- 6.2 Accept that scientific knowledge is always 'work in progress' and that something regarded as 'true' may be proven to be false, and vice-versa.
- 6.3 Are keen to acquire new knowledge, skills and abilities.
- 6.4 Are willing to share and discuss arguments, results and conclusions, including submitting one's own work to peer review.
- 6.5 Are convinced that academic debates should not be conducted on the basis of rhetorical qualities but that arguments must be considered and conclusions drawn on the basis of empirical results and valid criticism.
- G Students can critically discuss strong and weak points of their own research using fundamental techniques from the field of Big Data and Automated Content Analysis, and suggest improvements.
- H Students can critically reflect on the assumptions, strengths, limitations, and potential improvements of their own planned research using fundamental techniques from the field of Big Data and Automated Content Analysis. They can defend their methodological choices using empirical and methodological arguments and respond constructively to valid criticism.

1.3 Help with practical matters

While making your first steps with programming in Python, you will probably have many questions. Nevertheless, <https://google.com> and <https://stackoverflow.com> should be your first points of contact. After all, that is how we solve many programming problems ourselves...

It is also strongly encouraged to try to solve problems together with classmates. A successful technique can be *pair programming*, where two colleagues work together behind one screen (for an explanation, see https://en.wikipedia.org/wiki/Pair_programming).

If you cannot find a solution independently, you can pose questions during the Thursday lab sessions. Keep the remark in Section 2.5 in mind.

Chapter 2

Rules, assessments, and grading

The final grade for this course is composed of an individual in-class coding exercise (25%), an individual project pitch (10%), and a final individual project (65%).

In-class coding exercise (25%)

The Week 4 in-class coding exercise assesses students' individual ability to work with Python-based data-analysis workflows and their understanding of the fundamental concepts introduced during the first part of the course. Students are asked to read, adapt, test, debug, and/or explain code using fundamental techniques taught in the course. The exercise also includes short questions about relevant course literature. The exercise is completed individually and without the use of generative-AI tools.

Grading criteria are communicated to students together with the assignment. In general, the assessment considers:

- students' understanding of Python basics and fundamental course concepts;
- their ability to work independently with basic data-analysis code;
- the correctness and clarity of their code, explanations, and interpretations; and
- the quality of their responses to questions related to relevant course literature and methods.

Individual project pitch (10%)

In Week 7, students give an individual pitch of their proposed final project. The pitch assesses the fea-

sibility and methodological justification of the proposed research plan. It also assesses students' ability to justify methodological choices, acknowledge a relevant limitation or risk in the planned research, and respond constructively to critical questions.

Grading criteria are communicated to students together with the assignment. In general, the assessment considers:

- the clarity and feasibility of the proposed research project;
- the justification of the proposed data, research design, and analytical strategy; and
- the student's critical reflection on limitations or risks and ability to answer questions about the proposed work.

Final individual project (65%)

The final individual project is an individual computational communication-science research project. It accounts for 65% of the final grade and assesses students' ability to formulate and answer a research question using fundamental techniques taught in the course.

Detailed requirements concerning the report, project files, critical reflection, submission, attribution, and permitted generative-AI use are provided in Appendix A.

Grading and second attempt

Students must receive a passing grade (5.5 or higher) for each of the three individual assessment components: the in-class coding exercise, the individual project pitch, and the final individual

project. If the grade for one component is lower than 5.5, the student may complete one reassessment opportunity for that component, in the form and by the deadline communicated by the lecturer.

A reassessment of the in-class coding exercise is completed individually in a supervised setting and without generative-AI tools. A reassessment of the project pitch takes the form of an individual pitch or oral assessment. An improved version of the final individual project may not be graded higher than 6.0. If a reassessment is graded lower than 5.5, the course cannot be completed.

2.1 Presence and participation

Attendance is compulsory. Missing more than two meetings—for whatever reason—means that the course cannot be completed.

In addition to attending meetings, students are required to prepare the assigned literature and continue working on programming tasks after the lab sessions. Completing the course requires continuous self-study; attending lab sessions alone is not sufficient.

Active academic participation is expected throughout the course, including careful preparation, participation in discussions, and constructive engagement with classmates.

2.2 Staying informed

It is your responsibility to check the means of communication used for this course—your email account and, where applicable, Canvas or other platforms used by the lecturer—regularly, which in most cases means daily.

2.3 Plagiarism & fraud

Plagiarism is a serious academic violation. Cases in which students use material such as online sources or other sources in their written work and present this material as their own original work without citation or referencing will be reported to the Examencommissie of the Department of Communication.

If the committee concludes that a student has committed plagiarism, the course cannot be completed.

General UvA regulations concerning fraud and plagiarism apply; see also <https://student.uva.nl/en/content/az/plagiarism-and-fraud/plagiarism-and-fraud.html>.

2.3.1 Attributing code

Plagiarism rules apply to code in the same way as they apply to text. Just as academic writing builds on the work of other scholars, code may build on solutions developed by others or found in communities such as Stack Overflow. In submitted code, it must be transparent *what is your own work and what is not*.

There is no single citation style for code, but `#comments` can be used for this purpose, for example:

```
# The following function is copied from
https://stackoverflow.com/XXXXX/XXXXX
for an almost literal copy, or:
```

```
# The code in this cell is inspired by
https://...; I modified Y and Z
```

Make sure that there is no doubt about what is yours and what is from someone else.

This requirement also applies to code produced or substantially modified with the assistance of generative-AI tools. Such use must be disclosed as described in Section 2.3.2.

2.3.2 Use of generative AI tools

Generative-AI tools may support learning, but this course assesses your own ability to write, read, test, document, and interpret code and analyses. You remain responsible for all work submitted in your name.

1. **In-class coding exercise.** The Week 4 in-class coding exercise is individual work. Generative-AI tools may not be used during the exercise.
2. **Use of course data with generative AI.** You may send only small, de-identified code snippets or toy examples to external AI tools when you need help understanding or debugging Python. Do not upload substantial parts of project data, sensitive data, or data that

could enable re-identification. If you are unsure whether an excerpt is safe to share, do not upload it; ask the lecturer instead.

3. **Prohibited uses of generative AI for data and analysis.** Do not upload course datasets or full analysis notebooks to external generative-AI services. Do not use generative AI as a black-box analysis engine for graded work, for example by asking it to analyse your data or interpret a full set of results. Training, evaluating, and interpreting models must be done by you using Python and the tools taught in the course, unless the lecturer explicitly authorises another approach.
4. **Responsibility for code and lecturer support.** You are responsible for the correctness, efficiency, reproducibility, and documentation of all submitted code, regardless of whether generative AI was used. Teaching support focuses on code that reflects your own work and learning process; the lecturer may ask you to explain or adapt code before providing further assistance.
5. **Disclosure of generative-AI use in code.** Disclose any non-trivial AI contribution to code in the relevant notebook or script, for example in a Markdown cell or comment stating the tool used and its contribution. Normal plagiarism and fraud rules apply.

2.3.3 Working together

The in-class coding exercise, the individual project pitch, and the final project are *individual assessments*. Students may not collaborate on code, analysis, writing, slides, or other substantive parts of these assessments unless explicitly instructed otherwise.

During the Thursday lab sessions, collaboration and mutual support are strongly encouraged. Students are encouraged to discuss programming concepts, compare approaches, ask questions, and help one another work through exercises. This collaboration is intended to support learning and does not permit students to share or jointly produce assessed code, analyses, written project material, or pitch materials.

During the Week 7 project pitch, students are encouraged to discuss project plans and provide constructive feedback to peers. In some cases, overlap between final projects may be unavoidable, for example when students rely on similar data sources. If in doubt, discuss this with the lecturer in advance. It is almost always possible to find a solution if the issue is discussed before submission.

For all **ungraded** assignments and exercises, collaboration is explicitly encouraged. Experience with this course has shown that students who meet regularly with classmates to discuss weekly code examples and exercises often benefit from doing so.

2.4 Deadlines and handing in

The Week 4 in-class coding exercise is completed on Monday 16-11. The individual Week 7 project pitch is delivered on Monday 07-12.

For the final project, submit the report as a PDF file through Canvas and submit code, notebooks, data where permitted, and other required materials in one `.zip` archive. Do not email assessments directly.

Final projects that are not handed in on time will not be graded and receive the grade 1. This rule also applies to other assessed assignments. A deadline is met only when all required files have been submitted successfully.

2.5 Your computer and operating system: your responsibility

Python is a language, not a program, and there are many ways of running Python code.

It is difficult to provide individual support for issues specific to your computer, operating system, or keyboard. Things may work differently on Windows, macOS, and Linux. It is your responsibility to prepare your computer before the course starts. For example, a downloaded file may be located under `/home/damian/Downloads` on Linux, `/Users/damian/Downloads` on macOS,

or `C:\\Users\\Damian\\Downloads` on Windows.

Therefore, ensure that the following requirements are met before the course starts:

- Install a recent version of Python and ensure that you can run Jupyter Notebooks. Chapter 1 of van Atteveldt, Trilling, and Arcila Calderón (2022) provides instructions; you may skip the part about R. You can use Python natively or Anaconda.
- Ensure that you can install third-party packages. Test this by installing `pandas` and running `import pandas` without an error.
- Ensure that your keyboard produces straight quotation marks, `"`, rather than typographical quotation marks. Test this by running `print("hello world")` in a Jupyter notebook.
- Create a folder for this course in an easy-to-remember location, for example `/home/damian/bdaca` on Linux, `/Users/damian/bdaca` on macOS, or `C://Users//damian//bdaca` on Windows. Ensure that you can locate files in this folder from Jupyter Notebook. Preferably, do not use spaces in folder names.

If you do not succeed after trying these steps, ask classmates for help. If that does not resolve the issue, contact the lecturer as soon as possible.

Chapter 3

Schedule and Literature

The following schedule gives an overview of the topics covered each week, the obligatory literature that has to be studied each week, and other tasks the students have to complete in preparation of the class. In particular, the schedule shows which chapter of van Atteveldt et al. (2022) will be dealt with. Note that some basic chapters that explain how to install the software we are going to use have to be read before the course starts.

Next to the obligatory literature, the following books provide the interested student with more and deeper information. They are intended for the advanced reader and might be useful for final individual projects, but are by no means required literature. Bear in mind, though, that you may encounter slightly outdated examples (e.g., Python 2, now-defunct APIs etc.).

- McKinney, 2012: A lot of examples for data analysis in Python. A HTML-based version of the book can be viewed for free at <https://wesmckinney.com/book/>.
- VanderPlas, 2016: A more recent book on numpy, pandas, scikit-learn and more. It can also be read online for free on <https://jakevdp.github.io/PythonDataScienceHandbook/>. The contents are available as Jupyter Notebooks as well, see <https://github.com/jakevdp/PythonDataScienceHandbook>.
- The pandas cookbook by Julia Evans, a collection of notebooks on github: <https://github.com/jvns/pandas-cookbook>.
- Hovy, 2020: A thin book on bottom-up text analysis in Python with both a bit more math

background and ready-to-use Python code implementations.

- Salganik, 2017: Not a book on Python, but on research methods in the digital age. Very readable, and a lots of inspiration and background about techniques covered in our course.

Before the course starts: Prepare your computer.

✓ CHAPTER 1: INTRODUCTION

Make sure that you have a working Python environment installed on your computer before the course begins. Detailed installation instructions will be provided in advance. Students are responsible for bringing a laptop with a functioning Python environment to every meeting.

Week 1: Programming for Computational (Communication|Social) Scientists

Monday, 26–10. Lecture, 15:00–17:00

✓ KITCHIN (2014)

✓ HILBERT ET AL. (2019)

We discuss what Big Data and Computational (Social|Communication) Science are. We consider their opportunities and challenges, including implications for social-scientific research. We also introduce the tools used in Computational Social Science, particularly Python.

Additionally, the journal *Communication Methods and Measures* published a special issue (volume

12, issue 2–3) on Computational Communication Science. You are invited to read at least the editorial (van Atteveldt & Peng, 2018); other articles can be read later in the course.

Towards the end of the lecture, we will make first contact with writing and running Python code. We will also discuss expectations concerning reproducibility, responsible use of external code and AI tools, and active participation in the course.

Thursday, 29–10. Lab session, 15:00–17:00

✓ CHAPTER 3: PROGRAMMING CONCEPTS FOR DATA ANALYSIS

✓ CHAPTER 4: HOW TO WRITE CODE

You will receive a gentle introduction to computer programming. During the lab, you are encouraged to follow examples on your own laptop. We will take the first steps in Python and complete exercises on variables, data types, control flow, functions, and debugging.

Week 2: From files and APIs to lists, dictionaries, or dataframes

We discuss file formats such as `csv` and `json`; encodings; reading files into basic Python structures such as dictionaries and lists; reading data into dataframes; and retrieving data from local files, packages, and APIs. We also consider data provenance and the practical limits of data access.

Monday, 02–11. Lecture, 15:00–17:00

✓ CHAPTER 5: FROM FILE TO DATAFRAME AND BACK

✓ FREELON (2018-10)

This lecture provides a conceptual overview of different file formats and data sources, alongside practical guidance on handling data in basic Python and pandas. We introduce APIs as a means of data collection and discuss why documentation, metadata, and collection decisions matter for later analysis.

Thursday, 05–11. Lab session, 15:00–17:00

We will practise using the data structures introduced in Week 1 and work with different file formats. Exercises include inspecting data, handling encodings, reading nested JSON structures, and extracting information from a provided API response.

Week 3: Data wrangling and exploratory data analysis

You do not need Python to conduct conventional statistics. However, using a single environment for data acquisition, cleaning, analysis, and visualisation makes a workflow more transparent and reproducible. This week, we discuss how to organise data, including long and wide formats, and how to conduct exploratory analyses and create simple visualisations in Python.

Monday, 09–11. Lecture, 15:00–17:00

✓ CHAPTER 6: DATA WRANGLING

✓ CHAPTER 7.1: SIMPLE EXPLORATORY DATA ANALYSIS

✓ CHAPTER 7.2: VISUALISING DATA

We will learn how to prepare data for analysis and develop a first understanding of a dataset through exploratory analysis and visualisation. We cover data wrangling with pandas, including filtering, aggregating, reshaping between long and wide formats, joining datasets, and handling missing values.

Thursday, 12–11. Lab session and short lecture, 15:00–17:00

We will apply the techniques from the previous lectures to multiple datasets. The session also prepares students for the individual in-class coding exercise in Week 4. We conclude with a short introduction to text as data, including tokenisation and Bag-of-Words representations.

Week 4: Individual in-class coding exercise and introduction to textual data

Monday, 16–11. Individual in-class coding exercise, 15:00–17:00

This individual, on-campus coding exercise covers the foundations from Weeks 1–3. It assesses students’ ability to read, adapt, debug, and explain basic Python and data-analysis code and contributes 25% to the final grade.

Within the scheduled session, students have 60 minutes to complete the exercise, followed by a 10-minute submission buffer for saving, renaming, and uploading the notebook. The exercise is open book: students may consult lecture slides, readings, course notebooks, personal notes, and Python/pandas documentation saved on their computer or printed before the exercise. Internet access is permitted only to download the assignment and upload the completed notebook. Browsing websites, searching online, downloading other materials during the exercise, using generative-AI tools, and communicating with or receiving help from others are not permitted.

Thursday, 19–11. Lecture: Processing textual data, 15:00–17:00

- ✓ CHAPTER 9: PROCESSING TEXT
- ✓ CHAPTER 10: TEXT AS DATA
- ✓ CHAPTER 11, SECTIONS 11.1–11.3: AUTOMATIC ANALYSIS OF TEXT

Additional recommended background reading on stopwords: Nothman, Qin, and Yurchak (2018).

This session introduces text processing as a measurement problem. We explore tokenisation, lemmatisation, stopword removal, n-grams, word counts, word co-occurrences, and regular expressions. We implement selected techniques in Python and discuss how preprocessing choices affect the validity of automated content analysis.

Week 5: Unsupervised approaches to text analysis

This week, we transition from transparent frequency-based text representations to unsupervised machine-learning approaches. We discuss how traditional topic models relate to embedding-based methods and how these approaches can support exploratory text analysis.

Monday, 23–11. Lecture: From topic models to embeddings, 15:00–17:00

- ✓ CHAPTER 11.5: UNSUPERVISED TEXT ANALYSIS — TOPIC MODELLING AND BEYOND
- ✓ KROON, WELBERS, TRILLING, AND VAN ATTEVELDT (2024)
- ✓ MAIER ET AL. (2018)

We examine the logic of unsupervised text modelling and contrast traditional approaches such as LDA with modern embedding-based approaches. We introduce BERTopic as an example of a workflow that uses embeddings, dimensionality reduction, clustering, and topic descriptions.

We emphasise that model outputs are candidate patterns rather than established findings: they require substantive interpretation, inspection of representative documents, and evaluation of their robustness and limitations.

Thursday, 26–11. Lab session, 15:00–17:00

We apply an unsupervised text-analysis workflow to a textual dataset. We compare output from simpler approaches with embedding-based clustering and discuss interpretation, outliers, parameter choices, reproducibility, and limits of automated topic descriptions.

Week 6: Supervised approaches to text analysis

This week introduces the logic of supervised machine learning for automated content analysis. We focus on classification tasks, evaluation, and the difference between a technically functioning model

and a valid measurement of a communication-science construct.

Monday, 30–11. Lecture, 15:00–17:00

✓ CHAPTER 8: STATISTICAL MODELING AND SUPERVISED MACHINE LEARNING

✗ STUDENTS MAY SKIP SECTION 8.4: DEEP LEARNING

✓ BOUMANS AND TRILLING (2016)

We discuss supervised machine learning, labelled data, training and test data, classification, and performance evaluation. We consider accuracy, precision, recall, F1, and the importance of inspecting model errors. We also discuss how dictionary methods, conventional classifiers, embedding-based approaches, and LLM-assisted classification can be compared and validated.

LLM-assisted classification is discussed conceptually; the lab uses scikit-learn. This discussion does not authorise the use of generative AI as an analysis engine for graded work. Any practical use of LLM-assisted classification requires explicit authorisation from the lecturer and must comply with the course rules on data sharing and disclosure.

Thursday, 03–12. Lab session, 15:00–17:00

We work with scikit-learn to construct and evaluate a basic text classifier. Students will compare automated outputs with manually coded examples, inspect errors, and discuss what evidence would be needed to support a substantive interpretation.

Week 7: Project pitches, peer feedback, and consultation

Monday, 07–12. Individual project pitches and peer feedback, 15:00–17:00

Each student presents a short pitch for the final project. The pitch should address:

- the research question and substantive motivation;
- the planned dataset;

- the intended analytical strategy;
- preliminary data inspection or preliminary output;
- one anticipated issue concerning validity, ethics, reproducibility, or AI use, together with a feasible response or improvement.

The individual pitch contributes 10% to the final grade. After presenting, each student answers critical questions about the proposed data, methods, assumptions, and limitations. Students also provide structured written feedback to peers and use the feedback they receive to improve their project plans. These peer-feedback activities are formative and are not separately graded.

Thursday, 10–12. Individual consultation, 15:00–17:00

Students can consult individually with the lecturer about methodological, practical, ethical, or reproducibility questions concerning their final project.

Week 8: Assessment week

Final individual project

Deadline for handing in: Wednesday, 16–12, 23:59.

Appendix A

Final Individual Project

The final individual project accounts for 65% of the overall course grade. Students design and execute an independent Computational Communication Science project that demonstrates their ability to formulate a research question, collect and analyse data, and apply fundamental techniques taught in the course.

This is an individual assessment. The report and all submitted code must be the student's own work, except where external sources or permitted generative-AI contributions are appropriately disclosed in accordance with the course manual.

Your submission must include:

1. a PDF report; and
2. a `.zip` folder containing all relevant code and output files.

PDF report

The recommended length of the report is five to six pages (including references). It should include the following components.

A. Introduction (one to two pages)

- A clearly formulated research question;
- Any subquestions and/or hypotheses;
- References to relevant literature, including course readings where appropriate.

B. Analytic strategy

- A transparent overview of the methodological approach;
- A justification for the choice of techniques, referring explicitly to methods discussed during the course.

C. Dataset description

- A carefully collected and relevant dataset of non-trivial size, for example retrieved via APIs or constructed by combining existing datasets;
- An exploratory description of the dataset;
- A brief argument explaining why the dataset is suitable for the research question.

D. Results

- A clear presentation of the analyses and findings needed to answer the research question;
- Appropriate tables, figures, examples, or model-evaluation results;
- A substantive interpretation of the findings that distinguishes empirical results from speculation.

E. Conclusion and critical reflection

- A well-substantiated answer to the research question;
- A critical discussion of the strengths, limitations, and uncertainties of the research design, data, analysis, and conclusions;
- An explanation of the implications of these limitations for the interpretation of the findings, as well as feasible directions for future research.

The conclusion and critical-reflection component contributes to the assessment of students' ability to critically discuss the strengths and weaknesses

of their completed research and suggest improvements.

Project files (.zip folder)

Alongside the PDF report, include a compressed folder containing:

- Python scripts or notebooks for data collection, preprocessing, analysis, and visualisation. These files must be well documented, reproducible, and tailored to the student's own project;
- Output files, such as tables, plots, or summary statistics;
- The data needed to reproduce the analysis, where sharing those data is legally, ethically, and contractually permitted;
- A README file describing:
 - how to run the code;
 - what each script or notebook does; and
 - any software or library requirements.

Requirements and guidance

Depending on the chosen topic, students must apply at least one technique taught in the course. Projects should include an exploratory description of the dataset and apply fundamental analytical techniques.

During lab sessions and individual consultations, students may discuss:

- the scope of the project;
- the methodological requirements that the project must meet; and
- how the chosen analytical methods contribute to the final grade.

Important notes

- All external data sources and code, including libraries, templates, and tutorials, must be properly cited.
- Code must be tailored to the student's own project and run without errors.

- Any non-trivial contribution by a generative-AI tool must be disclosed in the relevant notebook or script, as specified in the course manual.
- The rules on generative-AI use in the course manual apply in full. In particular, students may not upload course datasets or full analysis notebooks to external generative-AI services, or use generative AI as a black-box analysis engine for graded work.
- Students are free to select any topic of interest—preferably topics within traditional communication science or the social sciences—provided that it is framed around a research question and based on original data analysis.

Submission instructions

Submit both the PDF report and the .zip folder containing code and outputs via Canvas.

Deadline: Submission is open until 16 December, 23:59. Late submissions are handled in accordance with the course manual and applicable examination regulations.

Literature

- Boumans, J. W., & Trilling, D. (2016). Taking stock of the toolkit: An overview of relevant automated content analysis approaches and techniques for digital journalism scholars. *Digital Journalism*, 4(1), 8–23. doi: 10.1080/21670811.2015.1096598
- Freelon, D. (2018-10). Computational research in the post-API age. *Political Communication*, 35(4), 665–668. Retrieved from <https://doi.org/10.1080/10584609.2018.1477506> doi: 10.1080/10584609.2018.1477506
- Hilbert, M., Barnett, G., Blumenstock, J., Contractor, N., Diesner, J., Frey, S., ... Zhu, J. J. H. (2019). Computational Communication Science : A Methodological Catalyzer for a Maturing Discipline. *International Journal of Communication*, 13, 3912–3934.
- Hovy, D. (2020). *Text analysis in Python for social scientists: Discovery and exploration*. Cambridge University Press. doi: 10.1017/9781108873352
- Kitchin, R. (2014). Big Data, new epistemologies and paradigm shifts. *Big Data & Society*, 1(1), 1–12. doi: 10.1177/2053951714528481
- Kroon, A. C., Welbers, K., Trilling, D., & Van Atteveldt, W. (2024, April). Advancing automated content analysis for a new era of media effects research: The key role of transfer learning. *Communication Methods and Measures*, 18(2), 142–162. Retrieved 2024-09-20, from <https://www.tandfonline.com/doi/full/10.1080/19312458.2023.2261372> doi: 10.1080/19312458.2023.2261372
- Lazer, D., Pentland, A., Adamic, L., Aral, S., Barabási, A.-L., Brewer, D., ... van Alstyne, M. (2009). Computational social science. *Science*, 323, 721–723. doi: 10.1126/science.1167742
- Maier, D., Waldherr, A., Miltner, P., Wiedemann, G., Niekler, A., Keinert, A., ... Adam, S. (2018). Applying LDA topic modeling in communication research: Toward a valid and reliable methodology. *Communication Methods and Measures*, 12(2-3), 93–118. doi: 10.1080/19312458.2018.1430754
- McKinney, W. (2012). *Python for data analysis*. O'Reilly.
- Nothman, J., Qin, H., & Yurchak, R. (2018). Stop Word Lists in Free Open-source Software Packages. In *Proceedings of workshop for nlp open source software (nlp-oss)* (pp. 7–12). Association for Computational Linguistics. doi: 10.18653/v1/W18-2502
- Salganik, M. J. (2017). *Bit by bit: Social research in the digital age*. Princeton University Press.
- Shah, D. V., Cappella, J. N., & Neuman, W. R. (2015). Big Data, digital media, and computational social science: Possibilities and perils. *The ANNALS of the American Academy of Political and Social Science*, 659(1), 6–13. doi: 10.1177/0002716215572084
- van Atteveldt, W., Trilling, D., & Arcila Calderón, C. (2022). *Computational analysis of communication: A practical introduction to the analysis of texts, networks, and images with code examples in python and r*. Wiley.
- van Atteveldt, W., & Peng, T. Q. (2018). When Communication Meets Computation: Opportunities, Challenges, and Pitfalls in Computational Communication Science. *Communication Methods and Measures*, 12(2-3), 81–92. doi: 10.1080/19312458.2018.1458084
- VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly.